Programming for ML and Data Science

Overview of ML and some tools

Prabhu Ramachandran

Department of Aerospace Engineering, IIT Bombay

2026-09-01

Overview of ML

- Very quick overview of key concepts
- More details in next semester

ML vs. AI

- AI: Artificial intelligence
- ML: Machine learning
- AI is more broad
- ML is an application (or part) of AI
- Will focus on ML
  - Specifically: **Regression** and **Classification**

Machine Learning

> A computer program is said to learn from experience E with respect to some class of tasks T, and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.
>
> Tom Mitchell (as cited by Kevin Murphy)

Scales of measurement

Stanley Smith Stevens' scales of measurement:

- Nominal
- Ordinal
- Interval
- Ratio

Different kinds

- Supervised learning
  - Classification
  - Regression
- Unsupervised learning
- Reinforcement learning

ML models: Learning a mapping

- Use as model → make predictions
  - Regression is a good example, $Y = f(X)$
  - Learn a model: $f(x) = \alpha + \beta x$
- Maybe a “black box” model or *interpretable*
- Could use probability theory internally

Supervised learning

- Learn a mapping, f from inputs $\mathbf{x} \in \mathcal{X}$ to outputs $\mathbf{y} \in \mathcal{Y}$
- Inputs are called *features*, *covariates*, or *predictors*
- Outputs called *label*, *target*, or *response*
- **Train** a model using given data $= \{(\mathbf{x_n}, \mathbf{y_n})\}_{n=1}^{N}$
- Training also called *model fitting*
- Many different ways to do this, different models/algorithms
- “Glorified curve fitting”

Example of terminology

- Finding A, B for linear model using regression: *training*
- Predicting new response based on new input: *inference*

Supervised learning: Regression

- Y is interval or ratio
- Learn the model: learn f in $Y = f(X)$
- Variety of different methods
  - Linear model
  - Polynomial
  - Support Vector Machine
  - Tree-based methods
  - DNN

Supervised learning: Classification

- Outputs: set of *labels* or *classess*: hence “classification”
- Y is categorical (nominal/ordinal)
- Also called *pattern recognition*
- Note: *binary classification* implies only two categories
- Example:
  - Given an image of an English letter, identify it
  - Given an image of an object, identify it
- Identify boundary between two different classes of parameters

Classification

- Logistic regression
- Tree based methods:
  - CART (classification and regression trees)
  - Random forest
  - G.B.M (Gradient boosted methods)
- SVC: Support vector classification
- Neural networks

How to train your AI?

- Loss function/risk minimization: $l(y, \hat{y})$
- Let $f(\mathbf{x}_n; \theta)$ be the learned mapping
- θ: set of parameters
- $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} l(y_i, f(\mathbf{x}_i; \theta))$
- Training finds the best parameters θ:
  - $\hat{\theta} = \underset{\theta}{\operatorname{argmin}}\, \mathcal{L}(\theta)$
- Find model parameters producing least loss

Regression

- Loss: $l_2(y, \hat{y}) = (y - \hat{y})^2$
- Mean Squared Error, MSE:
  - $\text{MSE}(\theta) = \frac{1}{N} \sum_{i=1}^{N} (y_i - f(\mathbf{x_i}; \theta))^2$
- $p(y|\mathbf{x}; \theta) = \mathcal{N}(y|\ f(\mathbf{x}; \theta), \sigma^2)$

Dealing with categoricals

- Encode the variable suitably
- Binary explanatory variable
  - Use an indicator variable (dummy variable)
- Nominal and Ordinal quantities
  - One-hot
  - Label encoding
  - Target encoding

Binary variable

- Variable is binary (or two variations)
- Encode as dummy variables (indicator function)
- Zero is usually chosen for the null/control variable
- β_0 is when the binary variable is 0
- $\beta_0 + \beta_1$ when it is 1

Multiple categories

- Similar to binary: but k values, take $k - 1$ dummies
- When $k - 1$ are zero: reference value
- Choose reference carefully as null/control when possible
- One-hot encoding!
- Use `pd.get_dummies(data=df, drop_first=True)`
- `sklearn.preprocessing.OneHotEncoder`

Label encoding

- Can encode the labels as integers
- More appropriate for ordinal data
- For example: Poor, Fair, Good, Excellent (etc.)
- Also note that the numbers should have equal gap
- Avoid this for linear models
- `sklearn.preprocessing.OrdinalEncoder`

Other encoding

- Target encoding: Replace each category by the average of the target
- Label encoder: better for decision trees

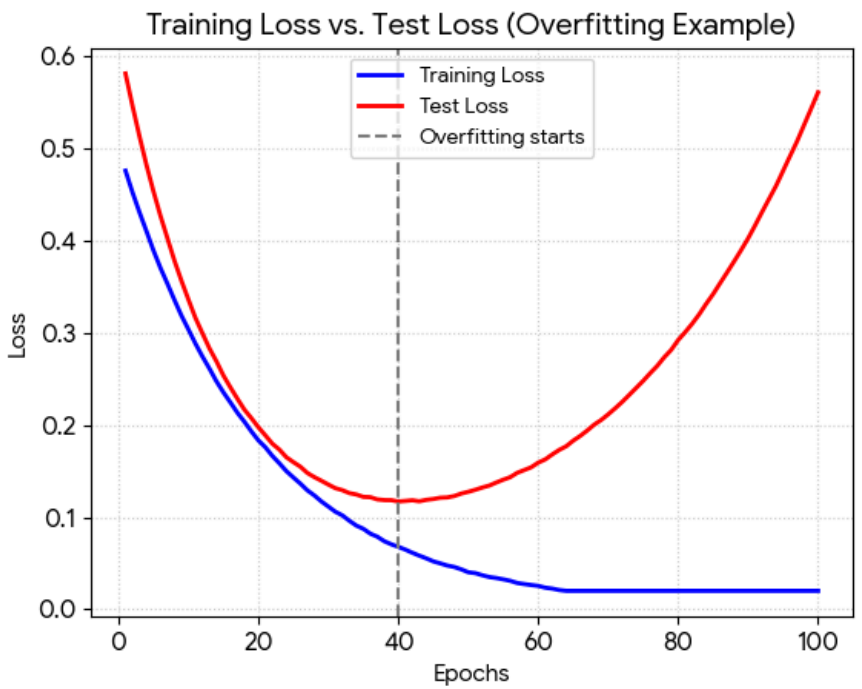
Parameters and hyperparameters

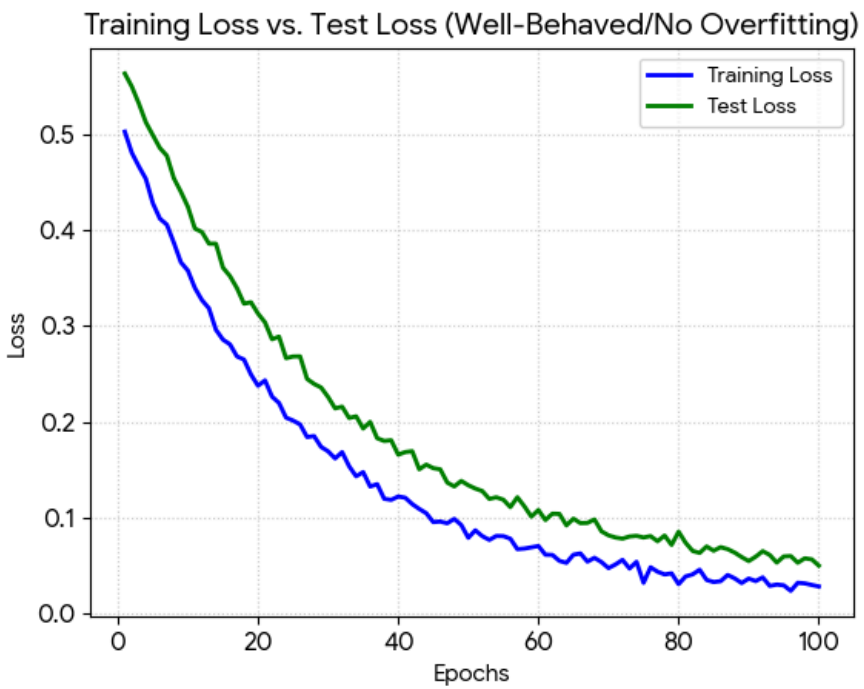
- Parameters
  - Typically relate to the model
  - Learned from data
- Hyperparameters
  - Relate to the learning algorithm (like learning rate)
  - Not usually learned from the data

Overfitting and generalization

- We must be wary of overfitting
- Do not want model to perfectly fit the training data
- We want the model to predict unseen data accurately
- Regularization helps
  - Minimize: $SS_R + \lambda||w||^2$
  - Ridge regression, Lasso, Elastic Net etc.
- Example: polynomial fitting

Example of overfitting





Model selection: assessing models

- Easy to create models
- Vitally important to test and assess them
- Use appropriate evaluative metrics

Data: Training, and Testing

- Algorithm can learn irrelevant things
- **Always** test it on **unseen data**
- Split data: training set ($\approx 75\%$), test set ($\approx 25\%$)
- Only learn from training set
- Assess with test set and check performance
- Restart process if performance is poor
- **Model should never learn from test data**
- `sklearn.model_selection.train_test_split`

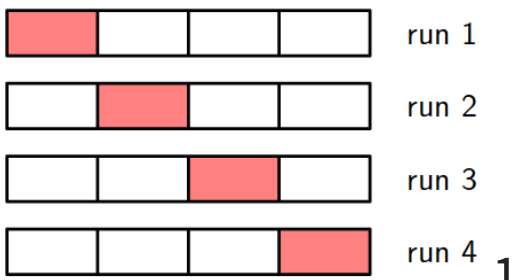
Data: Training, Testing, and Validation

- Training set (60%), Validation set (20%), Test set (20%)

1. Train for an epoch on training set
2. Estimate performance on validation set
3. Optimize hyperparams based on validation set
4. Once happy with the model check with test set

- Again, one should always test finally with the test set

Cross-validation



- Break data: training + test
- Break training data into N parts
- Each time, train with N-1; 1 for testing
- Average score for performance
- `sklearn.model_selection.cross_val_score`

General approach

1. Do you need regression or classification?
2. Use different methods
3. Evaluate metrics
4. Compare the methods
5. Choose appropriate method

Metrics for regression

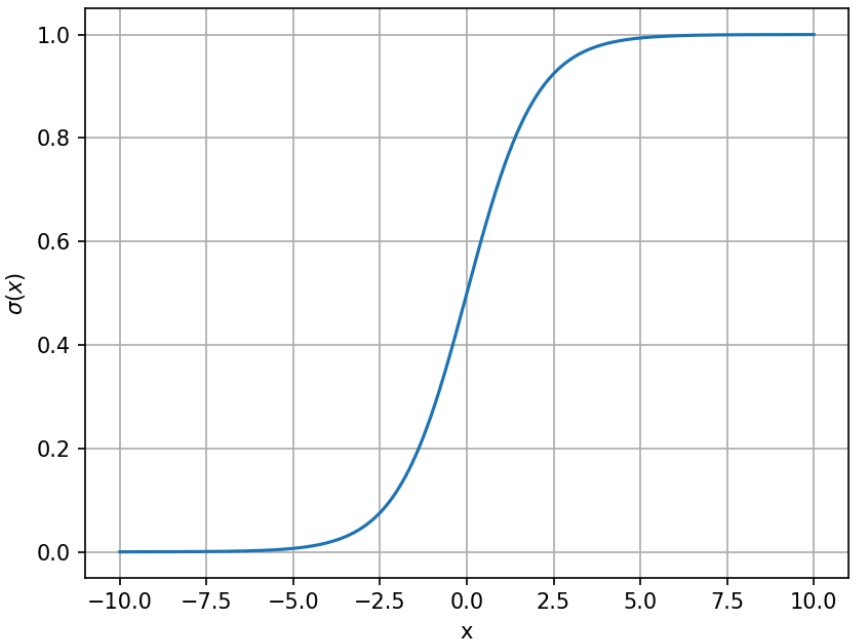
- Mean squared error
- R^2, adjusted R^2 (better)
- Distribution of residuals (ideally normal)
- QQ plot of residuals
- See scikit-learn metrics and scoring

Classification

- Recall that Y is categorical
- Different methods
- Only go into Logistic regression below

Logistic regression

- $y = \sigma(f(x))$
- y is discrete; x are features
- Sigmoid function applied on linear model
- Require **threshold** for switching point
- Boundary is straight if you use a linear model

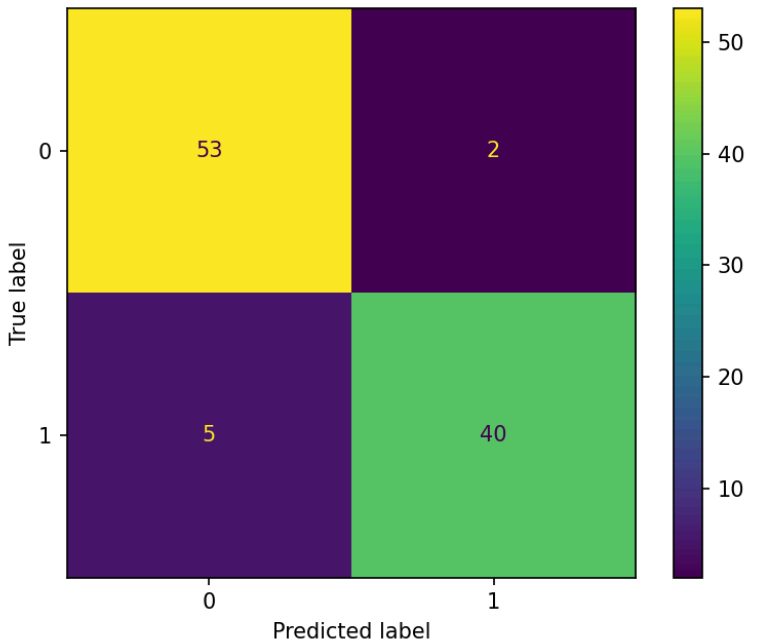


Example: SPAM filter for email

- Consider your emails
- An algorithm provides probability that email is SPAM
- $p = P(SPAM|email)$
- Let $\tau = 0.7$
- If $p < \tau$: OK
- If $p \geq \tau$: SPAM

Evaluation Metrics: Confusion matrix

- Confusion matrix
  - Predicted, Actual counts
  - TN: True Negative
  - TP: True Positive
  - FP: False Positive
  - FN: False Negative
- Multi-class: more rows/cols



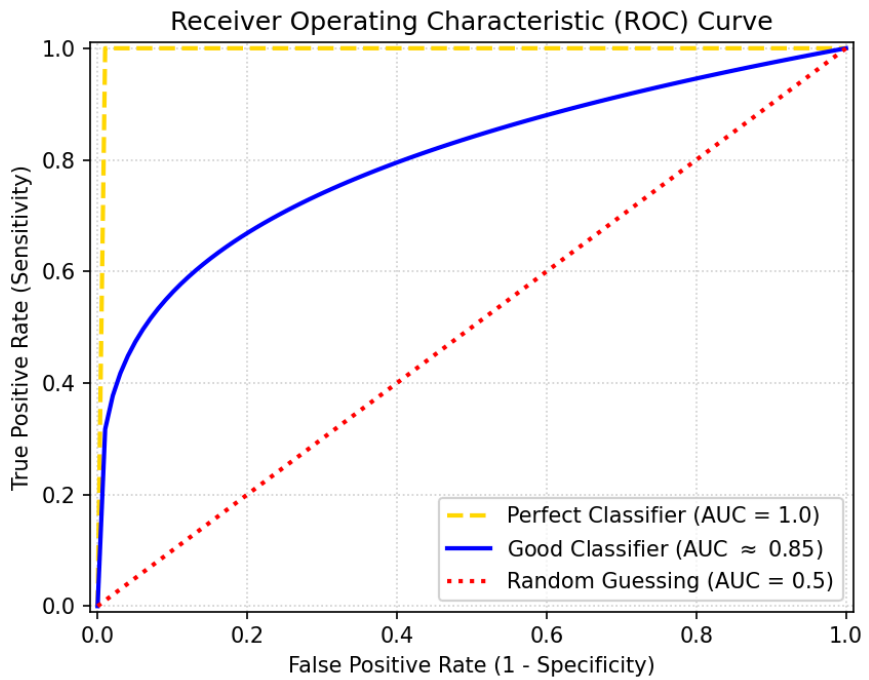
Evaluation Metrics

- Accuracy: (TP + TN)/total
- Precision: TP/(TP + FP)
- Recall: TP/(TP + FN)
- All should be close to 1: good classifier

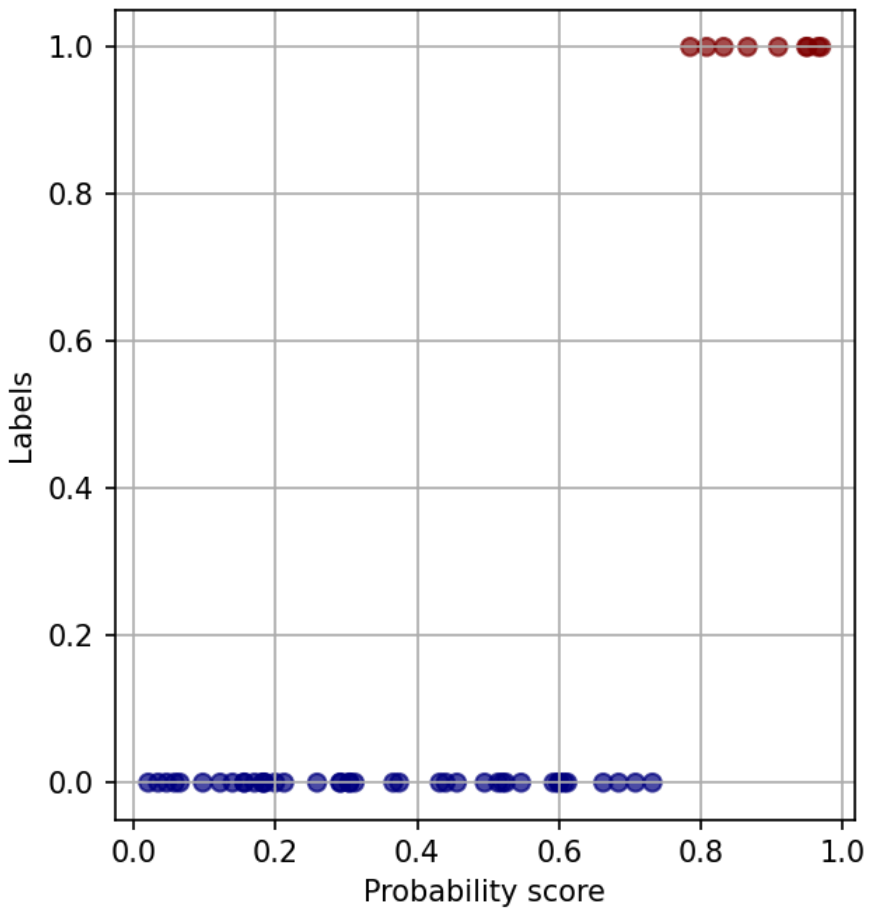
- True positive rate (TPR, Recall) : TP/(TP + FN)
- FPR: FP/(FP + TN)
- Accuracy not best measure esp. when classes imbalanced

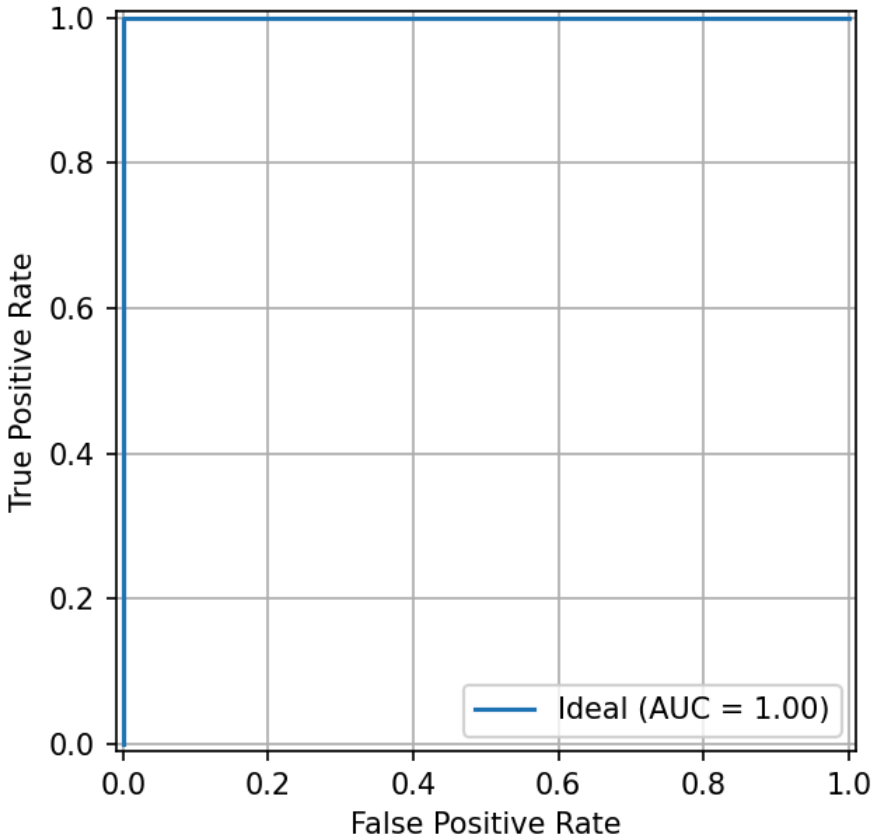
Evaluation metrics: ROC

- Receiver Operating Characteristic
- TPR vs. FPR
- Ex: marking emails as SPAM
- Area Under the Curve: AUC
  - Summarizes the ROC

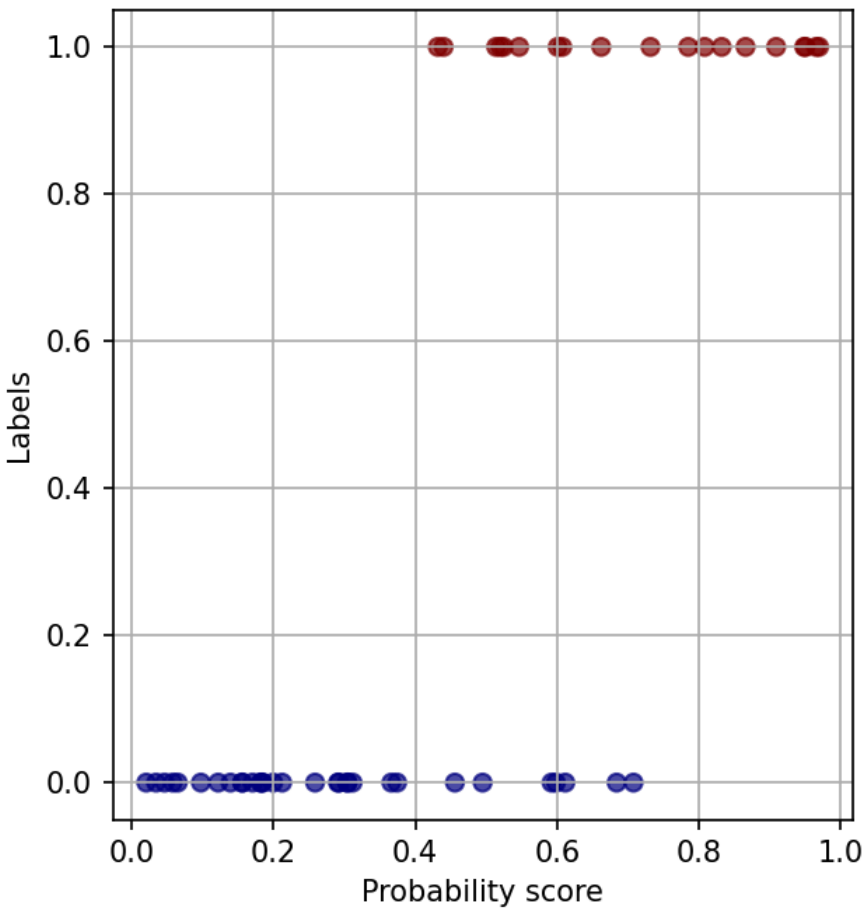


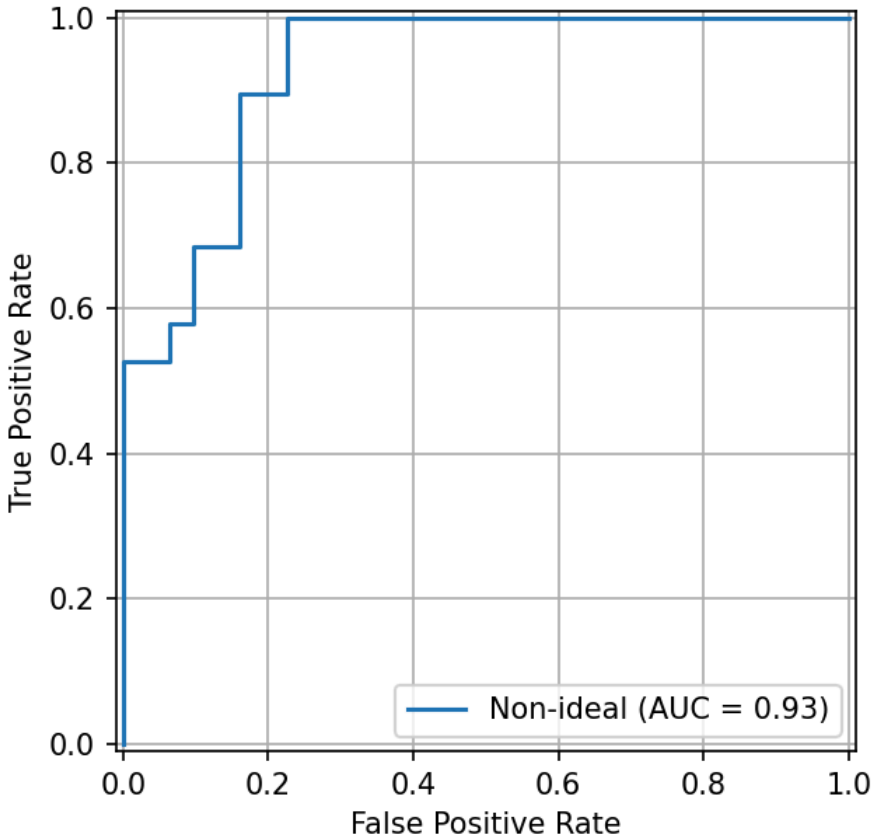
ROC: ideal classifier



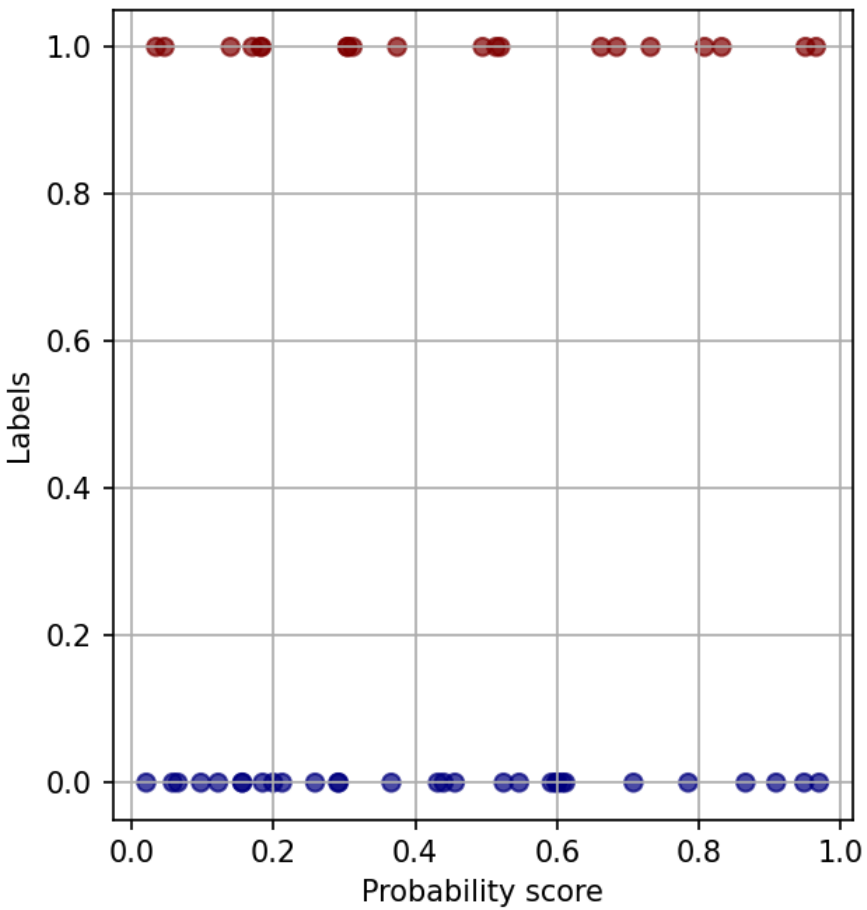


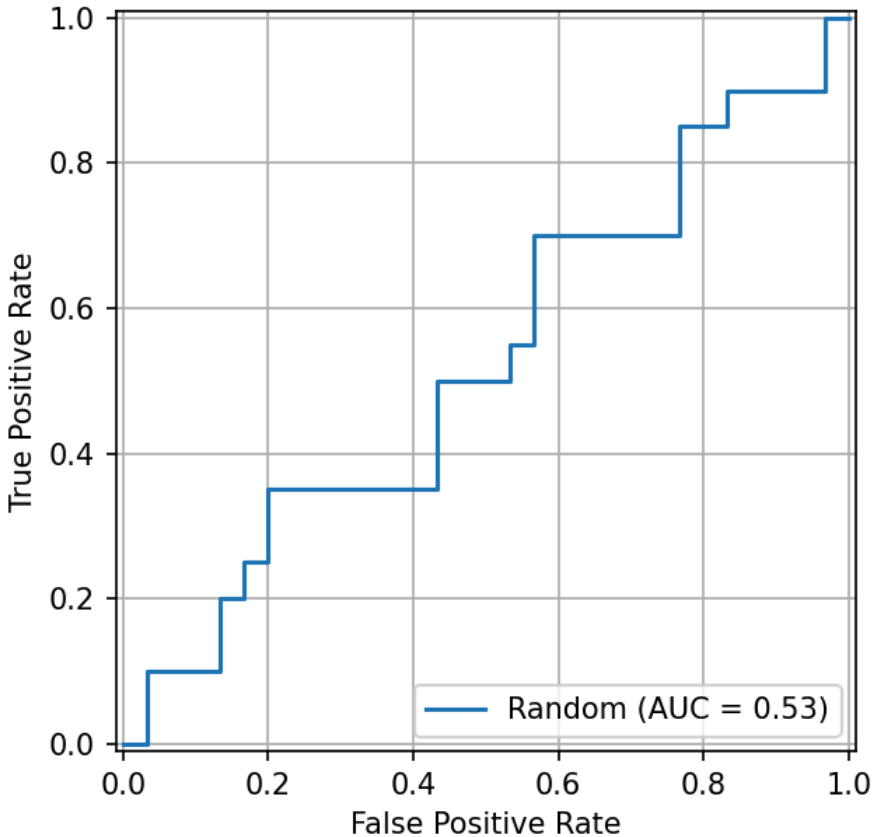
ROC: good classifier





ROC: lousy classifier





Evaluation metrics: F1 score

- F1 score:
  - Combines precision and recall: $1/P + 1/R = 2/F1$
  - Accuracy can be high but F1 low

Summary

- Overview of ML
- Supervised learning: Regression/Classification
- Training/Testing/Validation
- Metrics for assessing models